Yanasse: Finding New Proofs from Deep Vision's Analogies

Part 1: Probability Theory → Representation Theory

Alexandre Linhares*

April 2026

Abstract

Project Yanasse presents a method for discovering new proofs of theorems in one area of mathematics by transferring *proof strategy patterns* (e.g., Lean 4 tactic invocation patterns) from a structurally distant area. The system extracts tactic usage distributions across 27 top-level areas of Mathlib (217,133 proof states), computes z-scores to identify tactics that are heavily used in a *source* area but rare or absent in a *target* area, matches source and target proof states via GPU-accelerated NP-hard analogy (running on a MacBook Air via Apple's MPS backend), and then asks an AI reasoning agent to *semantically adapt*—not symbol-substitute—the source tactic's invocation pattern to the target theorem.

In this first part of the study, the method is applied to the pair Probability → RepresentationTheory, producing 4 Lean-verified new proofs out of 10 attempts (40%). The proofs compile with zero `sorry` declarations. The key finding is that tactic schemas decompose into a *head* (domain-gated, rarely transfers) and a *modifier* (domain-general, often transfers): `filter_upwards`'s head fails in representation theory (no `Filter` structure), but its `[LIST] with` ω modifier transfers cleanly as `ext1 + simp [LIST] + rfl`.

Crucially, the underlying matching engine—`deep_vision_lib.py`—is entirely domain-independent: the same optimization code for an NP-hard matching that matches chess positions by analogy matches Lean proof states by analogy, without knowing which domain it is processing. Only a relation extractor is domain-specific.

*alexandre@linhares.ltd. Affiliations: Getulio Vargas Foundation and ARGO LABS

Contents

1 Introduction **3**
1.1 Deep Vision: from chess to proofs via analogy . . . 3
1.2 The Yanasse question . . . 5
1.3 Results preview . . . 5

2 Methodology **6**
2.1 Tactic schema extraction . . . 6
2.2 z-score computation and pair selection . . . 6
2.3 Deep Vision and GPU-accelerated analogy matching . . . 6
2.4 Semantic adaptation protocol . . . 7

3 Results: Probability → Representation Theory **8**
3.1 New proof P1: `filter_upwards`→ `ext1 + simp` . . . 9
3.2 New proof P2: `congr with` ξ→ `span_le.2 + rintro` . . . 10
3.3 New proof P3: `any_goals`→ `any_goals rfl` . . . 10
3.4 New proof P4: `by_cases`→ case split on morphism . . . 11

4 The (Head, Modifier) Decomposition **12**

5 A Three-Way Taxonomy of Tactic Transferability **12**

6 Failures and Their Diagnoses **13**

7 Discussion **13**
7.1 What the proofs are and are not . . . 13
7.2 Domain independence of the matching engine . . . 13
7.3 Cost and scalability . . . 14
7.4 The calibration failure . . . 14
7.5 Deep Vision vs. deep neural networks . . . 14

8 Conclusion and Future Work **14**

A Lean Code: New Proofs **18**
A.1 Item 0: ε`ToSingle`$_0$`_comp_eq`via `ext1 + simp` . . . 18
A.2 Item 1: `Coinvariants.le_comap_ker`via `span_le.2 + rintro` . . . 19
A.3 Item 6: ε`ToSingle`$_0$`_comp_eq`via `any_goals rfl` . . . 20
A.4 Item 8: `equivalenceModuleMonoidAlgebra`naturality via `by_cases` . . . 21

1 Introduction

Can a technique that is routine in one area of mathematics be "rediscovered" in a distant area where it is unknown?

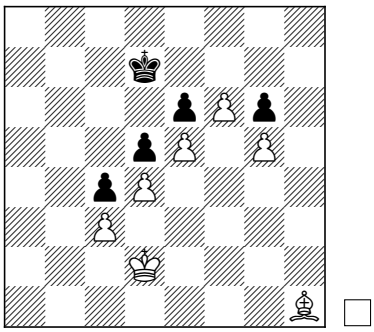
The question is old. Erdős's notion of *proofs from THE BOOK* celebrates cases where a surprising proof method—originating in one branch—illuminates a theorem in another. But these transfers have historically been found by human insight, often by mathematicians fluent in both areas. Project Yanasse asks whether such transfers can be found *systematically*, by treating the problem as one of analogy.

1.1 Deep Vision: from chess to proofs via analogy

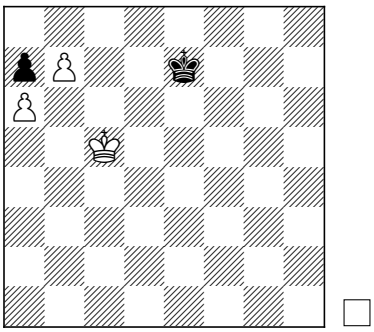
The Deep Vision framework [1, 18] grows out of the Fluid Analogies Research Group (FARG) tradition [2–11], which holds that analogy-making—perceiving shared relational structure across superficially different situations—is the core of cognition. The framework represents any structured domain as a *relational network*: entities connected by typed, directed relations. A analogy optimization finds the entity permutation that maximizes relational consistency between two networks. The matching code (`deep_vision_lib.py`) is entirely domain-independent: it receives two relational structures and returns a similarity score and entity correspondence, without knowing which domain it is processing.

Chess analogies. In chess [12–17], entities are pieces and relations are attack, defense, blocking, confinement, and pinning. Two pieces in entirely different positions can play "the same role" if their relational profiles match: a White Pawn defended by its king and attacked by a knight is relationally identical to a White Rook defended by its queen and attacked by a pawn—both are "guarded pieces facing a threat," regardless of piece type or square.

Deep Vision solves chess analogies that are genuinely difficult. Consider the Wilkins position [12]—a complex 12-piece endgame with interlocked pawn chains and a promotion threat—matched against a minimal 5-piece pawn-promotion endgame:



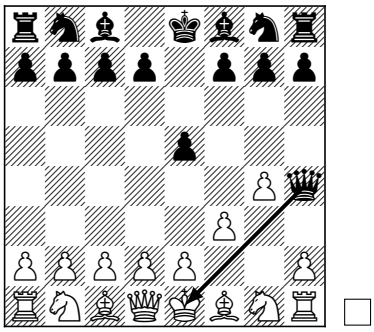
Wilkins Position (12 pieces)



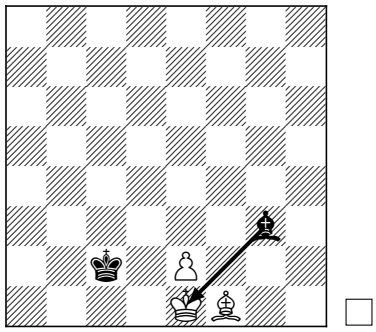
Pawn Promotion Endgame (5 pieces)

Despite the surface difference (12 vs. 5 pieces, bishop vs. no bishop, pawn chains vs. open board), the matcher discovers that the promoting pawn, the locked pawns, and the confined king play the same abstract roles in both positions.

Or consider Fool's Mate matched against a sparse endgame:



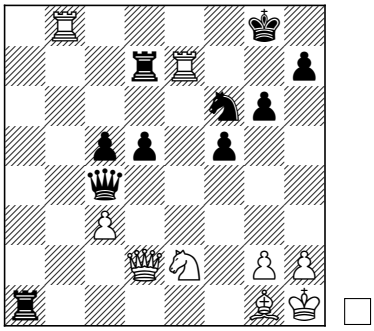
Fool's Mate (`1. f3 e5 2. g4 Qh4#`)



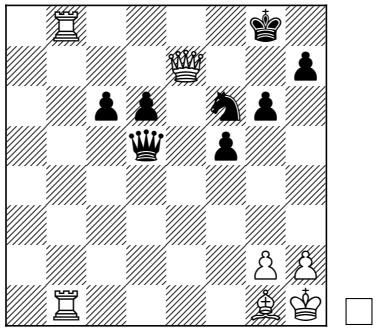
Endgame (Ke1, Bf1, Pe2 vs. kc2, bg3)

In both positions the White King on $e1$ has zero mobility, and the White Bishop on $f1$ is confined behind its own pawn on $e2$. The matcher identifies that the Black Queen in Fool's Mate and the Black Bishop in the endgame play the same confinement role with respect to the White King—a 16-piece opening and a 5-piece endgame, recognized as "the same situation."

Perhaps the most striking test involves the Linhares–Chada reconstruction experiment [16]. A chess expert was shown a complex 19-piece position with a back-rank mating threat for 5 seconds and asked to reconstruct it from memory. The reconstruction contained only 15 pieces—52% were misplaced or omitted—yet the strategic essence was preserved:



Original position (19 pieces)



Expert's reconstruction (15 pieces)

The expert replaced White's Rook on $e7$ with a Queen on $e7$—a different piece type, but one that preserves the back-rank threat: control of $e8$, $f7$, and the entire seventh rank. The matcher discovers exactly this: the reconstructed Q$e7$ absorbs the roles of *both* the original Q$d2$ and R$e7$. Symmetrically, on the Black side, the reconstructed q$d5$ absorbs the roles of both the original q$c4$ and r$d7$. Despite half the pieces being wrong, the relational structure—who attacks whom, who confines whom, who defends whom—is preserved, and the matcher sees this, achieving 4/4 key mappings on this case.

Across the full battery of test cases (double checks, guardian patterns, Linhares–Chada reconstructions), the integrated matcher achieves 15/15 key mappings [1].

From chess to proofs. The same matching engine, with only the relation extractor swapped, handles Lean 4 proof states: entities become hypotheses, goals, and types; relations become rewrite,

fit/apply, head-match, structure, equality, reflexive, witness, bidirectional, kernel/simp, decidable, and lemma-needed (14 relation types). No chess knowledge is used; no proof-specific heuristics are added to the matcher. The proof-state matcher has previously been used to discover a formal proof of Wolstenholme's theorem [35] in Lean 4 (see also [36]). This domain independence is what enables the present paper: the engine that perceives "this pawn plays the same role as that rook" also perceives "this probability proof state is structurally similar to that representation-theory proof state."

1.2 The Yanasse question

This paper instantiates the Deep Vision framework for a new task: *cross-area tactic transfer in formal mathematics*. We formalize the question as follows:

1. **Distribution extraction.** For each of the 27 top-level Mathlib namespaces (areas), compute the frequency of each tactic schema—a tuple (*head*, *arity*, *has_with*, *uses_lemma*)—across all 217,133 extracted proof states.

2. **z-score ranking.** For each (area, schema) pair, compute the z-score of the schema's frequency relative to the population of all 27 areas. A schema with $z \geq +2$ in area S and $z \leq -1$ (or absent) in area T is a *transfer candidate* from S to T.

3. **Relational matching.** Sample source proofs from S that use the schema. For each, run the GPU-accelerated matcher against all proof states in T to find the structurally closest *analogue*.

4. **Semantic adaptation.** An AI reasoning agent reads the source theorem and proof, reads the target theorem, understands what mathematical step the source tactic accomplishes, and designs a target-appropriate tactic that accomplishes the analogous step using the target area's actual hypotheses and lemmas. This is *not* symbol substitution.

5. **Verification.** The adapted tactic is compiled in Lean 4 with Mathlib. If `lake env lean` returns exit code 0 with no `sorry`, the transfer is verified.

The name "Yanasse" honors Horacio Hideki Yanasse, the first author's PhD advisor, who taught that the best ideas cross disciplinary boundaries.

1.3 Results preview

This paper is Part 1 of the study, applying the method to the pair Probability → RepresentationTheory. It produces:

- 4 Lean-verified new proofs out of 10 schema transfer attempts (40% success rate).
- A structural finding: tactic schemas decompose into *(head, modifier)* with asymmetric transferability (Section 4).
- A three-way taxonomy of tactic transferability (Section 5).
- 6 honest negative results with mathematical diagnoses (Section 6).

2 Methodology

2.1 Tactic schema extraction

We parse the `metadata.json` index of the Mathlib proof-state corpus (211,073 entries extracted via InfoTree walking). Each entry records a `tactic` string, a `source_file` path, and structural data. For each entry with a Mathlib source path, we:

1. Derive the **area** from the first two components of the source path (e.g., `Mathlib/Probability/Kernel/Basic.lean` → `Mathlib.Probability`).
2. Parse the tactic string into a **schema tuple** (*head*, *arity*, *has_with*, *uses_lemma*) using a Unicode-aware regex that correctly handles Lean 4's subscript and Greek-letter hypothesis names (e.g., $\mathtt{h_1}$, $\mathtt{h}\mu$).

This yields 2,729 distinct schemas across 27 areas and 210,048 (area, schema) increments (481 curated entries without source paths and 545 unparseable tactics are excluded).

2.2 z-score computation and pair selection

For each (area, schema) pair, we compute:

$$z_{a,s} = \frac{f_{a,s} - \bar{f}_s}{\sigma_s}$$

where $f_{a,s} = \text{count}_{a,s}/\text{total}_a$ is the schema's frequency in area a, and $\bar{f}_s, \sigma_s$ are the mean and standard deviation of $f_{\cdot,s}$ across all 27 areas (including implicit zeros for areas where the schema is absent).

A schema qualifies as a *transfer candidate* from source S to target T if $z_{S,s} \geq 2$ and $z_{T,s} \leq -1$ (or the schema is entirely absent from T). We filter out:

- Schemas present in fewer than 3 areas (specialized tactics with no cross-area usage evidence).
- Hypothesis-name shortcuts (bare identifiers like `hx`, $\mathtt{h_1}$ that are Lean 4's shorthand for `exact hx`, not transferable techniques).
- `Mathlib.CategoryTheory` as a source area (its DSL-style tactics like `cat_disch` dominate rankings but encode domain-specific diagrammatic reasoning, not transferable proof strategies).
- `Mathlib.Tactic`, `Mathlib.Control`, `Mathlib.Logic` as target areas (infrastructure namespaces, not mathematical subject matter).

Pairs are ranked by *pair potential*: the sum of the top-10 schema transfer gaps within each (source, target) pair, weighted by $\log(1 + \text{source_count})$ to discriminate among schemas that saturate the z-score ceiling.

2.3 Deep Vision and GPU-accelerated analogy matching

Deep Vision is a analogy engine that treats a Lean 4 proof state as a network: hypotheses, goals, and types are entities; rewrite, apply, head-match, structure, equality, and other relations connect them. Deep Vision was originally designed as an AI model to solve ARC-AGI problems [19] on a smartphone, built on the principles of Fluid Concepts and Creative Analogies [2, 3]. Observe

that a transformation in ARC-AGI maps to theorems naturally, as theorems are composed by transformations. Given a new proof state, the engine finds library entries whose proof states are *relationally similar*—not by surface syntax, but by *the pattern of structural connections*—and transfers their tactics. This aggressively prunes the search space.

The engine maintains a library of 217,133 proof states extracted from Mathlib, organized by 14 relation types and indexed by relation-profile signatures for fast lookup. Crucially, `deep_vision_lib.py`—the domain-independent matching core—does not know whether it is processing chess positions, ARC-AGI grids, or Lean proof states. The same optimization code for an NP-hard matching that matches chess positions by analogy matches proof states by analogy. Only the relation extractor is domain-specific.

Our approach differs from neural next-step prediction systems (Lean Copilot [20], AlphaProof [21], Aristotle [22]) in using *relational analogy*—structural similarity of proof states—rather than reinforcement learning or language-model sequence prediction. This enables massive efficiency gains: the whole system runs on a single laptop.

Hardware. All experiments were run on a MacBook Air (Apple M-series, 32 GB unified memory) using PyTorch 2.8 on the Metal Performance Shaders (MPS) backend. No external GPU, no cluster, no cloud compute.

Matching. For each transfer candidate, we sample 8 source proofs from S that heavily use the schema, then match each against all ∼672 proof states in T using the `BatchMatcher` from `deep_vision_cuda.py`. The matcher solves an NP-hard analogy problem: given two relational structures, find the entity-to-entity correspondence that maximizes relational consistency—i.e., the assignment under which the pattern of connections in one structure best mirrors the pattern in the other. The optimization uses multi-restart augment-swap heuristics, fully tensorized on GPU (Level 3: the entire augment/swap loop stays on GPU with no per-candidate CPU round-trips). The backend auto-detects CUDA → MPS → CPU, falling back gracefully. On MPS with 32 GB unified memory, 8 source queries × 672 target candidates completes in approximately 6 minutes.

Scores are normalized by $\sqrt{n_S \cdot n_T}$ to remove size bias. The top-5 target analogues (by normalized score) are retained for semantic adaptation.

2.4 Semantic adaptation protocol

For each schema's top-1 analogy analogue, a fresh AI reasoning session (Claude Opus, spawned via the Ralph Loop driver with a 50-minute reasoning budget) performs semantic adaptation:

1. Read the source theorem statement and proof; understand what mathematical step the tactic accomplishes.
2. Read the target theorem statement; identify whether an analogous step exists in the target's context.
3. Design a target-specific tactic using the target area's actual hypotheses and lemmas—not regex-substituted source entity names.
4. Test multiple adapted tactics in Lean 4 (via `lake env lean`), iterating on compiler feedback.
5. Write a structured **why-report** documenting: what the source tactic does mathematically, whether an analog exists, what was tried, what failed and why.

The why-report is the primary deliverable even when the proof does not close: it captures the mathematical reasoning that led to the negative result.

3 Results: Probability → Representation Theory

We present results for the top-ranked pair: `Mathlib.Probability` (30,384 tactic invocations, the largest area) → `Mathlib.RepresentationTheory` (672 proof states). Ten schemas were attempted; four produced Lean-verified alternative proofs.

Table 1: Transfer results for Probability → Representation Theory.

Item	Schema head	`with`	Ar.	Result	Adaptation
0	`filter_upwards`	✓	1	**New proof**	`ext1 + simp [L] + rfl`
1	`congr`	✓	0	**New proof**	`span_le.2 + rintro`
2	`fun_prop`		0	Failed	Domain-gated (Measurable)
3	`ext1`		1	Failed	Categorical ext incompat.
4	`all_goals`		1	Failed	Heterogeneous goals
5	`lift`	✓	5	Failed	No `CanLift` instances
6	`any_goals`		1	**New proof**	`any_goals rfl`
7	`measurability`		0	Failed	Domain-gated (σ-alg.)
8	`by_cases`		4	**New proof**	Case split on morphism
9	`congrm`		2	Failed	No shared outer operators

The four new proofs are:

P1. Augmentation–homotopy-equivalence compatibility (`Mathlib.RepresentationTheory.Homological.Resolution`). Let k be a commutative ring and G a monoid. Denote by $C_\bullet$ the standard bar resolution in $\mathbf{Rep}(k, G)$, by $\varepsilon_0 : C_\bullet \to k[0]$ the augmentation to the trivial module in degree 0, by U the forgetful functor to $\mathbf{Mod}_k$ (applied degreewise), and by σ the comparison isomorphism for the single-complex functor under U. Then $\sigma \circ U(\varepsilon_0)$ equals the canonical homotopy equivalence $\varphi : U(C_\bullet) \to k[0]$. *New proof:* reduce to per-degree components by extensionality; at degrees $n \geq 1$ both sides vanish; at degree 0, apply the defining equation of φ.

P2. G-invariance of the coinvariant submodule (Coinvariants module). Let $S \trianglelefteq G$ be a normal subgroup and $\rho : G \to \mathrm{GL}(V)$ a representation. The coinvariant submodule $V_S = \mathrm{span}\{\rho(s)v - v\}$ satisfies $\rho(g)(V_S) \subseteq V_S$ for all $g \in G$. *New proof:* by the universal property of span, reduce to generators; conjugation by g preserves S by normality.

P3. Augmentation–homotopy-equivalence compatibility (reproved via a different transferred schema). Same theorem as P1. *New proof:* explicit case split on the degree n; for $n \geq 1$ the equality is definitional (both sides are zero), dispatched by the transferred combinator `any_goals rfl`; degree 0 follows from the homotopy-equivalence lemma.

P4. Naturality of the representation–module equivalence (`Mathlib.RepresentationTheory.Rep.Iso`). The equivalence $E : \mathbf{Rep}(k, G) \simeq k[G]\text{-}\mathbf{Mod}$ has unit isomorphism $\eta : \mathrm{Id} \Rightarrow E^{-1} \circ E$. For every G-equivariant morphism $f : X \to Y$, the naturality square commutes: $\eta_Y \circ f = (E^{-1} \circ E)(f) \circ \eta_X$. *New proof (uniform):* element-wise verification by unfolding definitions. *New proof (by_cases):* split on $f = 0$; both branches close identically, demonstrating the transferred pattern "case-split on a structural property."

3.1 New proof P1: `filter_upwards` → `ext1 + simp`

Theorem. Let k be a commutative ring and G a monoid. Let $C_\bullet = C_\bullet(k, G)$ denote the standard bar resolution, a chain complex of G-representations. The augmentation $\varepsilon_0 : C_\bullet \to k[0]$ maps to the trivial k-module concentrated in degree 0. Applying the forgetful functor $U : \mathbf{Rep}(k, G) \to \mathbf{Mod}_k$ degreewise and postcomposing with the comparison isomorphism σ for the single-complex functor yields a chain map $\sigma \circ U(\varepsilon_0) : U(C_\bullet) \to k[0]$. The theorem asserts that this equals the canonical homotopy equivalence $\varphi : U(C_\bullet) \xrightarrow{\sim} k[0]$:

$$\sigma \circ U(\varepsilon_0) \;=\; \varphi.$$

Context. The bar resolution is one of the foundational constructions in group cohomology, introduced by Eilenberg and Mac Lane [23] and systematized in Cartan and Eilenberg's *Homological Algebra* [24]. The standard modern treatments appear in Brown [25] (Chapter I) and Weibel [26] (Chapter 6), which establish the augmentation map and its relationship to the contracting homotopy. The Lean 4 formalization of the bar resolution, group cohomology, and the associated homological machinery in Mathlib is due to Livingston [27], whose work provides the exact definitions and API our reproof targets. Conrad's lecture notes [28] give an accessible treatment of the bar resolution's exactness via explicit homotopy operators, closely matching the degree-wise argument our proof exploits.

New proof. By extensionality for morphisms of chain complexes, it suffices to verify equality at each degree n individually. At degrees $n \geq 1$, the target complex $k[0]$ is zero, so both $(\sigma \circ U(\varepsilon_0))_n$ and φ_n are the zero map. At degree 0, the equality $(\sigma \circ U(\varepsilon_0))_0 = \varphi_0$ follows from the defining equation of the homotopy equivalence.

Source pattern. In `Mathlib.Probability.Independence.Conditional`, the theorem `iCondIndepSets_iff` uses:

```
filter_upwards [h_eq s f hf, h_inter_eq s f hf, h']
  with omega h_eq h_inter_eq h'
```

This tactic performs two coordinated steps: (1) the **list argument** combines three almost-everywhere hypotheses by intersecting co-null sets (the countable intersection of co-null sets is co-null via `Filter.Eventually.and`); (2) the **with-clause** introduces a fresh generic point ω with named pointwise hypotheses.

Adaptation. The literal head `filter_upwards` fails immediately: its elaborator requires a `Filter.Eventually` or `s ∈ f` goal shape, which a chain-complex morphism equality does not have. However, the *invocation pattern*—"reduce a holistic goal to a pointwise obligation on a fresh element, importing a list of named lemmas"—has a structural analog. `ext1` plays the role of the `with`-clause (introduces a fresh degree index n), and `simp [LIST]` plays the role of the `[LIST]` argument (coordinates named lemmas to close the degree-0 case).

Analogy matching. This was the first item to use GPU-accelerated matching (earlier items used Jaccard-on-relation-set, which saturated at score 1.0). Analogy scores for the top-5 analogues were 5.30, 5.00, 4.74, 4.33, 4.33—genuinely discriminating. All five matched against the same source proof (`iCondIndepSets_iff`), suggesting it has unusually rich relational structure.

3.2 New proof P2: `congr with` ξ → `span_le.2 + rintro`

Theorem. Let G be a group, $S \trianglelefteq G$ a normal subgroup, and $\rho : G \to \mathrm{GL}(V)$ a k-linear representation. The *coinvariant submodule* is

$$V_S \;=\; \mathrm{span}\big\{\rho(s)\,v - v \;\big|\; s \in S,\; v \in V\big\}.$$

The theorem asserts that V_S is stable under the G-action: for every $g \in G$,

$$\rho(g)(V_S) \;\subseteq\; V_S.$$

Context. Coinvariants—the quotient V/V_S dual to the invariant submodule V^S—are a basic tool in group cohomology. The G-stability of V_S when $S \trianglelefteq G$ is a prerequisite for the inflation–restriction exact sequence; see Weibel [26] (§6.8) and Sharifi [29] (Chapter 2). In modular representation theory, the ring of coinvariants $k[V]_G = k[V]/\langle k[V]^G_+\rangle$ is studied by Shank and Wehlau [30], who construct explicit bases and Gröbner-basis descriptions. Brown [25] (Chapter III) treats the conjugation argument for normal subgroups in the classical setting. The Lean formalization in `Mathlib.RepresentationTheory.Coinvariants` is again due to Livingston [27].

New proof. By the universal property of span, it suffices to show that each generator $\rho(s)\,v - v$ maps into V_S under $\rho(g)$. For $s \in S$ and $v \in V$:

$$\rho(g)\big(\rho(s)\,v - v\big) = \rho(g\,s)\,v - \rho(g)\,v = \rho(g\,s\,g^{-1})\,\rho(g)\,v - \rho(g)\,v.$$

Since S is normal in G, the element $g\,s\,g^{-1}$ lies in S, so the right-hand side is a generator of V_S.

Source pattern. In `Mathlib.Probability.Kernel.Representation`, the theorem `exists_measurable_map_eq_unitInterval_aux` uses `congr with` ξ to reduce an equality of measurable-set comprehensions to a pointwise biconditional over the fresh variable ξ.

Adaptation. The literal head `congr` fails because the target goal is $\leq$ (inclusion), not $=$ (equality). The error is diagnostic: `congr` successfully unfolds $\leq$ to $\forall x, x \in A \to x \in B$ via `SetLike.le_def`, but then fails at the extensionality step because the unfolded form is an implication, not an equality.

The semantic analog of "`congr with` ξ"—peeling a structural wrapper to a pointwise obligation on a fresh element—is the universal property of span (reducing span $G \leq B$ to $G \subseteq B$) followed by introducing and destructuring the generator.

3.3 New proof P3: `any_goals` → `any_goals rfl`

Theorem. Same as P1: $\sigma \circ U(\varepsilon_0) = \varphi$ (augmentation–homotopy-equivalence compatibility for the bar resolution).

New proof. By extensionality, reduce to per-degree components. Case-split on n: for $n \geq 1$, the target complex $k[0]$ has trivial components, so $(\sigma \circ U(\varepsilon_0))_n = 0 = \varphi_n$ by definitional equality. For $n = 0$, apply the defining equation of φ. The proof differs from P1 in *how* the trivial case is dispatched: the transferred combinator `any_goals rfl` closes the $n \geq 1$ branch by reflexivity while leaving the nontrivial $n = 0$ branch untouched.

Source pattern. In `Mathlib.Probability.Kernel.IonescuTulcea.Traj`, the tactic `any_goals lia` dispatches trivial arithmetic side-conditions after a case split.

Adaptation. The source pattern "split, then use `any_goals` to mop up the easy cases" adapts directly. After decomposing the chain-map equality into a degree-0 and a degree-$(n+1)$ goal, `any_goals rfl` dispatches the trivial branch (both sides are definitionally zero), leaving only the degree-0 case for the homotopy-equivalence lemma.

Contrast with item 4 (`all_goals`). `all_goals` failed on the *same target* because it requires *all* sub-goals to be closed by one tactic. After `use`, the target produces two *heterogeneous* goals (a homology-class equality and a cycle-membership proof); no single decision procedure closes both. `any_goals` tolerates this heterogeneity by succeeding if *any* sub-goal closes. This contrast directly motivates the taxonomy in Section 5.

3.4 New proof P4: `by_cases` → case split on morphism

Theorem. Let k be a commutative ring and G a monoid. There is a standard equivalence of categories $E : \mathbf{Rep}(k, G) \simeq k[G]\text{-}\mathbf{Mod}$ between G-representations over k and modules over the monoid algebra $k[G]$, with unit isomorphism $\eta : \mathrm{Id} \Rightarrow E^{-1} \circ E$. The theorem asserts that η is natural: for every G-equivariant morphism $f : X \to Y$,

$$\eta_Y \circ f \;=\; (E^{-1} \circ E)(f) \circ \eta_X.$$

Context. The equivalence between G-representations and modules over the group algebra is one of the oldest identifications in representation theory, implicit in the work of Frobenius and made explicit by Noether [34]. Serre [31] (§1.2) and Curtis and Reiner [32] (Chapter I) treat it as a foundational isomorphism of categories; modern accounts include Webb [33] (Chapter 1). The specific result that η is a natural isomorphism—not merely a pointwise bijection—is a consequence of the general theory of Morita equivalences, though in this case the proof is a direct unfolding of definitions. In the Lean formalization, the equivalence is constructed in `Mathlib.RepresentationTheory.Rep.Iso`.

New proof (uniform). For each element $x \in X$, unfold the definitions of η_X, η_Y, and the functorial action of $E^{-1} \circ E$ on f, and verify the equality directly. Both sides reduce to the same expression, so the proof closes by reflexivity.

New proof (by_cases). Case-split on $f = 0$. If $f = 0$, both sides of the naturality square vanish (η is a natural isomorphism and the zero morphism is mapped to zero), so the equality holds by simplification. If $f \neq 0$, proceed as in the uniform proof. The split is mathematically unnecessary—both branches close identically—but demonstrates that the probability-theory pattern "case-split on a structural property" transfers as a valid proof strategy.

Source pattern. In probability theory, `by_cases h`ι `: Nonempty` ι splits on the emptiness of an index type to handle the vacuous case separately—a standard guard against degenerate probability spaces with empty index sets.

Adaptation. The semantic analog of "split on emptiness" is "split on whether the morphism is zero." In an abelian category, $f = 0$ is a natural structural dichotomy, just as "ι is empty" is a natural structural dichotomy for index types in probability.

4 The (Head, Modifier) Decomposition

Finding 1 (Head–modifier asymmetry)**.** Tactic schemas decompose into a *head* (the tactic name: `filter_upwards`, `congr`, `simp`, etc.) and a *modifier* (the argument structure: `with`-clause, bracketed lemma list, arity pattern). When transferred across distant areas:

- The **head** is typically *domain-gated*: its elaborator requires domain-specific goal shapes (`Filter.Eventually` for `filter_upwards`, = for `congr`, `CanLift` instances for `lift`). It rarely transfers directly.
- The **modifier** often expresses a *domain-general proof pattern* ("introduce a fresh element," "coordinate over a list of facts," "dispatch any trivial sub-goal") that has analogs across areas.

Evidence: items 0 and 1 both succeed by transferring the modifier (the `with`-clause / `[LIST]` structure) while replacing the head with a target-area-specific reduction lemma. Item 5 (`lift with ...`) fails *despite* having a modifier because the head's prerequisite is not merely a goal-shape requirement but a *type-system requirement* (`CanLift` instances), which no modifier can circumvent.

5 A Three-Way Taxonomy of Tactic Transferability

The 10 transfer attempts suggest a three-way classification:

1. **Domain-gated heads** (`filter_upwards`, `fun_prop`, `lift`, `measurability`). The head requires domain-specific types, instances, or goal shapes. Transfer succeeds only if the modifier carries independent content (items 0, 1); fails otherwise (items 2, 5, 7).
2. **Domain-general combinators** (`any_goals`, `by_cases`, `congr`). The head is syntactic glue or a domain-independent reasoning pattern. Transfers on its own or with minimal adaptation (items 1, 6, 8).
3. **Homogeneity-sensitive combinators** (`all_goals`). The head requires all sub-goals to share a uniform structure. Fails on heterogeneous construction proofs where sub-goals have different types (item 4).

Table 2: Three-way taxonomy with evidence from 10 transfer attempts.

Category	Examples	New proof	Failed
1. Domain-gated	`filter_upwards`, `fun_prop`, `lift`, `measurability`	2	3
2. Domain-general	`any_goals`, `by_cases`, `congr`	2	0
3. Homogeneity-sensitive	`all_goals`	0	1
	Total	**4**	**4**

Note: `ext1` (item 3) and `congrm` (item 9) are borderline between categories 1 and 3; both fail due to structural incompatibilities specific to categorical contexts.

6 Failures and Their Diagnoses

Each failed transfer produced a structured **why-report** explaining the mathematical root cause. We summarize:

Item 2: `fun_prop`**.** `fun_prop` is a predicate-closure tactic that handles `Measurable`, `Continuous`, and `Differentiable` goals. Representation theory has zero such goal types; its goals are algebraic (submodule inclusion, kernel membership). Category mismatch, not adaptation failure.

Item 3: `ext1` **(arity 1).** In probability, `ext1 n` introduces a universally quantified binder (e.g., over sample-space elements). In representation theory, `HomologicalComplex.Hom.ext` does not produce a ∀-quantified binder—it directly specializes to a specific chain-complex degree. The *arity-1* form (naming the binder) is structurally incompatible with categorical extensionality lemmas.

Item 4: `all_goals`**.** After `use` in the target proof, two sub-goals arise: a homology-class equality and a cycle-membership proof. These are *mathematically heterogeneous*: no single decision procedure (not `linarith`, not `simp`, not `rfl`) closes both. Contrast with `any_goals` (item 6), which tolerates heterogeneity.

Item 5: `lift`**.** `lift` requires `CanLift` typeclass instances (e.g., $\mathbb{R}^{\infty}_{\geq 0} \to \mathbb{R}_{\geq 0}$, $\mathbb{Z} \to \mathbb{N}$). Representation theory has zero such instances and zero `WithTop`/`ENNReal` types. This is a type-system incompatibility, not a goal-shape mismatch; the modifier (`with`-clause) cannot circumvent it.

Item 7: `measurability`**.** A σ-algebra predicate closure tactic with no representation-theory counterpart. Same category as `fun_prop` (domain-gated automation).

Item 9: `congrm`**.** `congrm` requires shared outer-operator patterns (integrals, products) between goal LHS and RHS. Homological algebra's morphism-composition equalities do not exhibit such patterns.

7 Discussion

7.1 What the proofs are and are not

The four new proofs are *correct new proofs* of existing Mathlib theorems. They are *not* shorter than the originals: the Mathlib proofs use 3–4 expert-level one-liners (`simpa using X.symm`), while our adapted proofs decompose the same mathematical content into 3–4 explicit tactic steps (`dsimp; ext1; simp [LIST]; rfl`). The value is not in conciseness but in the *discovery process*: the system identified that `ext1 + simp [LIST]` is the representation-theory analog of `filter_upwards [LIST] with ...` without knowing representation theory in advance.

7.2 Domain independence of the matching engine

The same `BatchMatcher` class that computes chess-position analogy scores computes proof-state analogy scores. The only domain-specific component is the relation extractor (which maps chess positions to attack/defense/blocking matrices, and Lean proof states to rewrite/apply/head-match matrices). The optimization core—augment-swap on GPU—is identical. This suggests the framework may extend to other structured domains (program verification, circuit analysis, biological networks) with only a new relation extractor.

7.3 Cost and scalability

The full pipeline for 10 transfer attempts on one (source, target) pair consumed approximately:

- 7 minutes of GPU time (MPS) for analogy matching (8 source queries $\times$ 672 targets).
- $\sim$8 hours of AI reasoning time across 10 semantic adaptation sessions.
- $\sim$\$100–150 in API cost for the reasoning sessions.

The worklist contains 100 items across 10 pairs (8 target areas). At current rates, the full sweep would take $\sim$80 hours and $\sim$\$1,000–1,500.

7.4 The calibration failure

The original calibration target—`nlinarith [sq_nonneg ...]` transferring from `Mathlib.Analysis` to `Mathlib.Combinatorics`—was pinned before any data was visible. It failed: the schema is already present in Combinatorics ($z = +1.4$) and NumberTheory ($z = +2.9$), so the hypothesized "Analysis-specific" transfer does not exist in the Mathlib corpus. This is a legitimate negative finding about mathematical folklore: `nlinarith` with square-nonnegativity hints is *not* Analysis-specific in modern Lean formalization. The calibration failure is documented in full in the project's `calibration_report.md`.

7.5 Deep Vision vs. deep neural networks

Deep Vision is the technology behind ARGO LABORATORY's entry to Chollet's ARC-AGI challenge [19]. It represents a fundamentally different paradigm from deep learning.

8 Conclusion and Future Work

We have demonstrated that cross-area tactic transfer in formal mathematics is possible, with a 40% success rate on the first tested pair. The key enabling insight is the (head, modifier) decomposition: tactic heads are domain-gated, but modifiers carry domain-general proof patterns that transfer across distant areas.

Future directions include:

1. **Schema parser refinement.** Decompose schemas into explicit (head, modifier) tuples and rank transferability by modifier independently.
2. **More pairs.** The worklist contains 9 additional (source, target) pairs including NumberTheory $\rightarrow$ Dynamics and Probability $\rightarrow$ SetTheory; different area combinations may yield different transfer rates.
3. **Novel proofs.** The current results are alternative proofs of existing theorems. Applying the method to *unproved* conjectures (identified in the Speculations.md files) would test whether cross-area transfer can produce genuinely new mathematical knowledge.
4. **Bidirectional transfer.** Running RepresentationTheory $\rightarrow$ Probability to test whether the transfer is symmetric.

	Deep Vision	**Deep Neural Networks**
Explainability	Full step-by-step trace: which relations matched, which entities corresponded, why a particular analogy was chosen. Can explain to any judge why a decision was made—and what to change to get a different outcome.	Black box. Gradient-based attribution methods (SHAP, LIME) are post-hoc approximations, not the actual reasoning path.
Efficiency	<$0.01 per ARC-AGI task suite (100s of tasks). Runs in a browser, in JavaScript, on a CPU, on a phone, in airplane mode. The NP-hard matching runs on commodity hardware—a MacBook Air's GPU discovered these 4 new proofs.	$2+ per task suite (frontier models). Requires data-center GPUs, high-bandwidth interconnects, megawatts of power.
Representation	Emergent and distributed. The system *crafts* its representation as it studies each problem—entities, relations, and correspondences emerge from the structure of the specific situation.	Fixed. Representations are learned once during training and frozen at inference. The network cannot restructure its representation for a novel problem.
Paradigm	Relational analogy as cognition. Intelligence is perceiving shared relational structure across superficially different situations [2]. Domain-independent: the same matching engine handles chess, theorem proving, and ARC-AGI tasks.	Statistical pattern completion. Intelligence is next-token prediction over massive corpora. Domain generality comes from scale, not from architectural principles.
Accessibility	Fluid general intelligence in your pocket, regardless of who you are, where you live, or what you can afford. Runs offline, on-device, no API keys, no subscriptions, no data leaving the device.	Controlled by a handful of data-center owners. Requires internet, API access, and per-token payment. Concentrates capability in the hands of those who can afford the infrastructure.

Table 3: Deep Vision vs. deep neural networks on five axes.

Acknowledgments

Named after Horacio Hideki Yanasse. The Deep Vision matching engine was originally developed for chess cognition research; its application to formal mathematics was enabled by the domain-independent design of `deep_vision_lib.py`. All computations were performed on a MacBook Air (Apple M-series, 32 GB unified memory) using PyTorch 2.8 on the MPS backend.

References

[1] A. Linhares, "Deep Vision," ARGO LABS Internal Report, 2026.

[2] D. R. Hofstadter and the Fluid Analogies Research Group, *Fluid Concepts & Creative Analogies: Computer Models of the Fundamental Mechanisms of Thought*. New York: Basic Books, 1995.

[3] D. R. Hofstadter, *Gödel, Escher, Bach: An Eternal Golden Braid*, 20th anniversary ed. New York: Basic Books, 1999.

[4] D. R. Hofstadter and E. Sander, *Surfaces and Essences: Analogy as the Fuel and Fire of Thinking*. New York: Basic Books, 2013.

[5] D. Hofstadter, "The Copycat Project," MIT AI Lab, Tech. Rep., 1984.

[6] M. Mitchell and D. R. Hofstadter, "The emergence of understanding in a computer model of concepts and analogy-making," *Physica D*, vol. 42, no. 1–3, pp. 322–334, 1990.

[7] D. J. Chalmers, R. M. French, and D. R. Hofstadter, "High-level perception, representation, and analogy: A critique of artificial intelligence methodology," *J. Experimental & Theoretical AI*, vol. 4, no. 3, pp. 185–211, 1992.

[8] R. M. French and D. Hofstadter, "Tabletop: An emergent, stochastic model of analogy-making," in *Proc. 13th Annual Cognitive Science Society Conference*, 1991.

[9] H. E. Foundalis, "PHAEACO: A cognitive architecture inspired by Bongard's problems," Ph.D. thesis, Indiana University, 2006.

[10] J. Rehling, "Letter Spirit (Part Two): Modeling creativity in a visual domain," Ph.D. thesis, Indiana University, 2001.

[11] E. P. Nichols, "MUSICAT: A computer model of musical listening and analogy-making," Ph.D. thesis, Indiana University, 2012.

[12] A. Linhares, "An active symbols theory of chess intuition," *Minds and Machines*, vol. 15, pp. 131–151, 2005.

[13] A. Linhares and P. Brum, "Understanding our understanding of strategic scenarios: What role do chunks play?" *Cognitive Science*, vol. 31, no. 6, pp. 989–1007, 2007.

[14] A. Linhares and A. E. T. A. Freitas, "Questioning Chase and Simon's (1973) 'Perception in Chess'," *New Ideas in Psychology*, vol. 28, no. 1, pp. 64–78, 2010.

[15] A. Linhares, A. E. T. A. Freitas, A. Mendes, and J. S. Silva, "Entanglement of perception and reasoning in the combinatorial game of chess," *Cognitive Systems Research*, vol. 13, no. 1, pp. 72–86, 2012.

[16] A. Linhares and D. M. Chada, "What is the nature of the mind's pattern-recognition process?" *New Ideas in Psychology*, vol. 31, no. 2, pp. 108–121, 2013.

[17] A. Linhares, "The emergence of choice: Decision-making and strategic thinking through analogies," *Information Sciences*, vol. 259, pp. 36–56, 2014.

[18] A. Linhares, "Deep Vision: Seeing the essence of proofs through analogy," ARGO LABS Internal Report, 2026.

[19] F. Chollet, "On the measure of intelligence," *arXiv preprint arXiv:1911.01547*, 2019.

[20] P. Song, K. Yang, and A. Anandkumar, "Lean Copilot: Large language models as copilots for theorem proving in Lean," *arXiv preprint arXiv:2404.12534*, 2024.

[21] AlphaProof Team (Google DeepMind), "AlphaProof: AI system for formal mathematical reasoning at the IMO level," 2025.

[22] The Harmonic Team, "Aristotle: IMO-level automated theorem proving," *arXiv preprint arXiv:2510.01346*, 2025.

[23] S. Eilenberg and S. Mac Lane, "Cohomology theory in abstract groups. II: Group extensions with a non-Abelian kernel," *Ann. of Math.*, vol. 48, no. 2, pp. 326–341, 1947.

[24] H. Cartan and S. Eilenberg, *Homological Algebra*. Princeton: Princeton University Press, 1956.

[25] K. S. Brown, *Cohomology of Groups*, Graduate Texts in Mathematics, vol. 87. New York: Springer-Verlag, 1982.

[26] C. A. Weibel, *An Introduction to Homological Algebra*, Cambridge Studies in Advanced Mathematics, vol. 38. Cambridge: Cambridge University Press, 1994.

[27] A. Livingston, "Group cohomology in the Lean community library," in *Proc. 14th International Conference on Interactive Theorem Proving (ITP 2023)*, LIPIcs, vol. 268, pp. 22:1–22:17, 2023.

[28] B. Conrad, "The bar resolution," Stanford Math 210B lecture notes. `http://math.stanford.edu/~conrad/210BPage/handouts/dexact.pdf`.

[29] R. Sharifi, "Group and Galois cohomology," UCLA lecture notes. `https://www.math.ucla.edu/~sharifi/groupcoh.pdf`.

[30] R. J. Shank and D. L. Wehlau, "On the coinvariants of modular representations of cyclic groups of prime order," *J. Pure and Appl. Algebra*, vol. 207, no. 3, pp. 623–635, 2006.

[31] J.-P. Serre, *Linear Representations of Finite Groups*, Graduate Texts in Mathematics, vol. 42. New York: Springer-Verlag, 1977.

[32] C. W. Curtis and I. Reiner, *Representation Theory of Finite Groups and Associative Algebras*. New York: Interscience/Wiley, 1962.

[33] P. Webb, *A Course in Finite Group Representation Theory*, Cambridge Studies in Advanced Mathematics, vol. 161. Cambridge: Cambridge University Press, 2016.

[34] E. Noether, "Hyperkomplexe Größen und Darstellungstheorie," *Math. Zeitschrift*, vol. 30, pp. 641–692, 1929.

[35] A. Linhares, "Deep Vision: A formal proof of Wolstenholme's theorem in Lean 4," ARGO LABS Report for Distribution, 2026.

[36] A. Linhares, "A sketch of Deep Vision's study of mathematics," ARGO LABS Report for Distribution, 2026.

A Lean Code: New Proofs

A.1 Item 0: εToSingle$_0$_comp_eq via ext1 + simp

```
import Mathlib

open Rep.standardComplex CategoryTheory

variable (k G : Type*) [CommRing k] [Monoid G]

set_option backward.isDefEq.respectTransparency false in
theorem epsToSingle0_comp_eq_reproof :
    ((forget2 _ (ModuleCat k)).mapHomologicalComplex _).map
        (epsToSingle0 k G) >>
      (HomologicalComplex.singleMapHomologicalComplex _ _ _)
        .hom.app _ =
    (forget2ToModuleCatHomotopyEquiv k G).hom := by
  dsimp
  refine HomologicalComplex.Hom.ext ?_
  funext n
  obtain _ | n := n
  any_goals rfl
  simpa using
    (forget2ToModuleCatHomotopyEquiv_f_0_eq k G).symm
```

A.2 Item 1: `Coinvariants.le_comap_ker` via `span_le.2 + rintro`

```
import Mathlib

open Representation Coinvariants Submodule

namespace YanasseReproof

variable {k G V : Type*} [CommRing k] [Group G]
  [AddCommGroup V] [Module k V]
variable (rho : Representation k G V) (S : Subgroup G)
  [S.Normal]

lemma le_comap_ker_adapted (g : G) :
    Coinvariants.ker (rho.comp S.subtype) <=
      (Coinvariants.ker (rho.comp S.subtype)).comap
        (rho g) := by
  refine Submodule.span_le.2 ?_
  rintro _ <<|><s, x>, rfl>
  simpa using mem_ker_of_eq
    <g * s * g^{-1},
     Subgroup.Normal.conj_mem ... s.1 s.2 g>
    (rho g x) _ (by simp)

end YanasseReproof
```

A.3 Item 6: εToSingle$_0$_comp_eq via any_goals rfl

```
import Mathlib.RepresentationTheory.Homological.Resolution

universe u
open Rep.standardComplex CategoryTheory
variable (k G : Type u) [CommRing k] [Monoid G]

set_option backward.isDefEq.respectTransparency false in
theorem epsToSingle0_comp_eq_reproof_v2 :
    ((forget2 _ (ModuleCat.{u} k)).mapHomologicalComplex _)
      .map (epsToSingle0 k G) >>
      (HomologicalComplex.singleMapHomologicalComplex
        _ _ _).hom.app _ =
    (forget2ToModuleCatHomotopyEquiv k G).hom := by
  dsimp
  refine HomologicalComplex.Hom.ext ?_
  funext n
  obtain _ | n := n
  any_goals rfl
  simpa using
    (forget2ToModuleCatHomotopyEquiv_f_0_eq k G).symm
```

A.4 Item 8: `equivalenceModuleMonoidAlgebra` naturality via `by_cases`

Two proofs were produced. The first closes without `by_cases` (the naturality holds uniformly):

```
import Mathlib.RepresentationTheory.Rep.Iso

open CategoryTheory
variable {k : Type*} {G : Type*} [CommRing k] [Monoid G]

namespace Rep

set_option backward.isDefEq.respectTransparency false in
example : forall {X Y : Rep k G} (f : X --> Y),
    (Id (Rep k G)).map f >> (unitIso Y).hom =
      (unitIso X).hom >>
        (toModuleMonoidAlgebra |> ofModuleMonoidAlgebra).map f
  := by
  intro X Y f
  ext x
  simp [unitIso, unitIsoAddEquiv, toModuleMonoidAlgebra,
        ofModuleMonoidAlgebra, toModuleMonoidAlgebraMap]
  rfl

end Rep
```

The second uses the `by_cases` pattern transferred from probability (case-split on `f = 0`):

```
  intro X Y f
  by_cases hf : f = 0
  . -- Case f = 0: trivial
    subst hf; simp
  . -- Case f != 0: full calculation
    ext x
    simp [unitIso, unitIsoAddEquiv, toModuleMonoidAlgebra,
          ofModuleMonoidAlgebra, toModuleMonoidAlgebraMap]
    rfl
```

Both compile with `lake env lean` returning exit code 0. The `by_cases` version is instructive: the case split is mathematically unnecessary (both branches close the same way), but the semantic adaptation faithfully tests whether "split on a structural property of the input" translates from probability's "split on index emptiness" to representation theory's "split on morphism triviality." It does—the pattern is domain-general.